Student B number: `B300238`

Hi!
Well done for all the effort that you have put into doing BPSM Optional ICA1!
There were two things that needed to be completed for this Optional ICA as given on https://bioinfmsc8.bio.ed.ac.uk/AY25_BPSM_OPTIONAL_ICA1.html

1. **50% of the "marks"**: a **PUBLIC** GitHub repository called MyFirstPipeline containing a `ccrypt` encrypted tar.gz file called B*xxxxxx*-2025.MyFirstPipeline.tar.gz.cpt, as detailed in the BPSM git and GitHub lecture.
   The tar.gz.cpt file should **only** contain the pipeline programme/code for me to run on our server, as well as the full git log of this coding project (in a `all_the_things_I_did` file).
   The pipeline programme/code itself should contain lots of "comments" so than anyone looking at the code knows what each bit is doing
2. **50% of the "marks"**: a PDF file called B*xxxxxx*-2025.MyFirstPipeline.pdf, submitted via Learn Dropbox, that:
   - gives the link for your **PUBLIC** GitHub MyFirstPipeline repository (e.g. https://github.com/*Bxxxxxx*-2025/MyFirstPipeline)
   - gives the `ccrypt` encryption key to open the B*xxxxxx*-2025.MyFirstPipeline.tar.gz.cpt file
   - has an overview/flowchart that describes how the pipeline processes the data
   - briefly indicates why you have chosen the programme parameters/flags that you have for each step
   - indicates any things the user will have to do to make things work (hopefully very few!)
   - highlights any difficulties, if any, that you have come across
   - indicates any additional/alternative features that you think might be beneficial to include

---

Code comments

- **Could I clone the GitHub repository asked for?**
  Yes, no problems!
- **Date/time on GitHub repo**

```
git log | grep Date
```

```
Date:   Fri Oct 10 13:24:29 2025 +0100
```

- **Could I decrypt the file with the password given?**
  Yes, worked fine!
- **Did the GitHub repository only include the pipeline code as a tar.gz filefile, as requested?**
  Yes

```
tar tvfz B300238-2025.MyFirstPipeline.tar.gz
```

```
-rwxr-xr-x s2906787/g_s2906787 1615 2025-10-10 13:09 alignment_HISAT2.sh
-rw-r--r-- s2906787/g_s2906787  379 2025-10-10 13:18 all_the_things_I_did
-rwxr-xr-x s2906787/g_s2906787 1472 2025-10-10 13:09 countreads_bedtools.sh
-rwxr-xr-x s2906787/g_s2906787 2971 2025-10-10 13:09 groupings.sh
-rwxr-xr-x s2906787/g_s2906787  875 2025-10-10 13:09 meancount_per_gene.sh
-rwxr-xr-x s2906787/g_s2906787  288 2025-10-10 13:09 myfirstpipeline.sh
-rwxr-xr-x s2906787/g_s2906787 1281 2025-10-10 13:09 QC_FAST.sh
-rwxr-xr-x s2906787/g_s2906787  965 2025-10-10 13:09 QC_trimming_bbduk.sh
```

- **Was your user name visible anywhere?**

```
grep Author *things* | cut -d ' ' -f2,3 | sort | uniq
```

```
B300238-2025 <email_not_provided>
```

```
git log | grep Author | cut -d ' ' -f2,3 | sort | uniq
```

```
B300238-2025 <email_not_provided>
```

No, couldn't find it at all: well done!

- **What did the git log suggest to me?**

```
egrep "Date" *things* | awk '{if(NF>1){print $2,$3,$4}}' | uniq -c
```

```
egrep -v "Auth|Date|commit" *things* | awk '(NF>1)'
```

Dates

```
2 Fri Oct 10
```

Messages

```
Updated 10 Oct 25: Edited MyFirstPipeline shell scripts
```

Was git actually used during coding at all? It really doesn't look like it, unless you have done all the coding over a space of 2 minutes or so. The messages also should be informative. Doing this properly was an important component of the Optional ICA.

Like most normal users, I did not read the manual first... to see if the programme is intuitive to use...

- **Could I run the script directly as a script (did it have a shebang line?)?**
  Yes, but I had to work out which one to use! If you have multiple scripts, do make it clear which one should be run first, so that everyone could work it out, e.g. `run_me.sh` to start the process... and ideally, make the first one start the second one once it has finished, and so on...
  Did you write do all these in Windows, rather than nano/vi? Is that why you have NO GIT/GITHUB??? PLEASE don't do it this way...!

```
find . -type f -name "*.sh" -exec sed -i 's/\r$//' {} \;
```

```
# Is this safe to do!?
./QC_FAST.sh && \
  ./QC_trimming_bbduk.sh && \
  ./alignment_HISAT2.sh && \
  ./countreads_bedtools.sh && \
  ./groupings.sh && \
  ./meancount_per_gene.sh
```

- **Did the script ask me where I wanted to put files while it was running/when it was finished?**
  No, I never got asked: it assumed I wanted to put it where I was running it from, which was fine this time.

- **Could I run the script directly in my home space (i.e. no links to places I can't get to easily without immediately knowing who you are)?**
  Yes, seemed to find everything.
- **Did the code use the `Tco2.fqfiles` content to set up loops for pretty much all steps?**

```
grep -c Tco2.fqfiles *.sh
```

```
alignment_HISAT2.sh:0
countreads_bedtools.sh:0
groupings.sh:3
meancount_per_gene.sh:0
myfirstpipeline.sh:0
QC_FAST.sh:0
QC_trimming_bbduk.sh:0
```

```
cat *.sh | grep Tco2.fqfiles
```

```
# First, copy raw Tco2.fqfiles to groupings dir
cp /localdisk/data/BPSM/MyFirstPipeline/fastq/Tco2.fqfiles ./groupings
done < ./groupings/Tco2.fqfiles
```

No, `Tco2.fqfiles` was not used much for this (but had obviously been read, as sample groups were identified...); this would NOT work with the full dataset with samples from other groups, or additional replicates, or filenames that don't end 1.fq.gz

```
# All the details were provided in a structured format, seven fields per line
```

```
head -n 3 Tco2.fqfiles
```

```
SampleName      SampleType      replicate       Time    Treatment       End1    End2
Tco230  Clone1  1       0       Uninduced       Tco-230_1.fq.gz Tco-230_2.fq.gz
Tco935  Clone1  1       24      Induced Tco-935_1.fq.gz Tco-935_2.fq.gz
```

These can be extracted as variables, as the file is parsed, line by line, using a `while` loop something like this

```
while read SampleName SampleType replicate Time Treatment End1 End2
do
echo "Processing sample ${SampleName}..."
# do cool stuff here
done < ${my_Tco_detailsfile}
```

loop.

You could use this loop numerous times to do different things, that is absolutely fine!
For example, the sample group for each sample can be automatically derived by combining fields (variables) obtained from the relevant fields (columns) of `Tco2.fqfiles` which in turn could be used to generate file names, make folders, in fact, pretty much anything/everything you needed.

We used a very similar sort of approach when we processed our BLAST outputs in Lecture07.

- **Could I have run the script if there had been additional samples?**
  No, sadly, wasn't sufficiently generic: use the `Tco2.fqfiles` sample details file more carefully: just need to count the number before calculating the means! It wouldnt deal with samples from, for example, Clone 3.
- **Did the code have lots of comments so I could see what the code was doing?**
  Yes, sufficient, a few more would have been OK
- **Was it logically laid out?**
  Yes
- **Did it use variables, or were most things hard-coded?**
  Used variables, but perhaps then looked at the contents, rather than just deciding on what could be done WITH the contents, e.g. what was in each column, what it meant.
  Used variables but some critical things (sample groups/sequence name parts) were "hard-coded" too?!
- **Did it use loops, or were most things hard-coded?**
  Used loops, but perhaps nowhere near as much as they should have been!
- **Did it use things that we hadn't covered in the course, like bash functions?**
  No, but that doesn't matter here, I was just curious.
- **Was there evidence that a flow-chart or similar had been used in planning the design (i.e. output from one bit being the input for the next)?**
  Yes, clearly planned in advance, flow chart shown in the PDF indicates you understood how outputs can also be inputs.
- **Were the replicates kept separate?**
  Yes, they are replicates! We might need them to do statistical analyses later on...

Were the commands correct?

- **initial processing stages**
  DIDN'T use info file `Tco2.fqfiles` ...
  You used variables as references to places/files: good!
  You do NOT need to copy ANY files...
  You do NOT need to unzip ANY files....
- **fastqc**
  `fastqc` accepts wild cards, and can be multi-threaded, so could/should have run `fastqc` on all samples with one command: much faster and more efficient.
  `fastqc` also works on gzipped files, so there is no need to unzip OR move the fastq files.
  A summary table of all samples would be very useful to look at, and saved as a record for the future? If one was generated, I wasn't told when it was running.
  I saw little or no processing of the `fastqc` outputs and it would have been good to get a chance to look at them. I do not really know what I had got (how many reads IN EACH one; the information IS in the data file produced by fastqc, we want more than just "Per base sequence quality" pass/warn/fail! The outputs were not very comprehensive: `fastqc` produces a lot of info, AND html files...: were the samples any good at all?!
  It went on to do trimming, was it needed? Which ones got trimmed?
  How about running `firefox` to display (some of) the outputs?
- **Were there problems using `firefox` (if used)?**
  User not shown any html outputs
- **Was the user given an option to quit after viewing the QC**
  No, analysis continued irrespective of how "good" or "bad" the data were.
  We did learn how to get user input in the lectures:

```
# Maybe we could ask the user what country to process?!
```

```
read -p "What country shall we process the data for? " wantedcountry
```

```
What country shall we process the data with?
```

- **bowtie2 or hisat2 alignment**
  One could have used the `Tco2.fqfiles` for this step too...
  Were any of the samples poor quality in terms of alignments.... if so, which one was it...? It was Tco-17 that was REALLY bad, and Tco-580 was probably marginal/worriesome too. I didnt see any assessment of the alignments: the code carried on regardless...
  Genome index building: reference does not need to be unzipped, but if it was, it should be gzipped up again after (or just deleted locally, if you copied it)
  Would probably recommend you use MANY more than 1 thread for genome indexing and/or the alignment steps; `nproc` will show you how many we have on our server... and it is MUCH faster! Remember the big fruit salad problem?!

- **bedtools**
  possibly incorrect parameter used with `bedtools` which made your analyses afterwards much harder?
  `bedtools multicov *.bam` will put all the counts data in one output file in bamfile name order, which makes the subsequent processing a lot easier, and means `bedtools` is only run once (it's not very fast)!
  You chose to do it one by one rather than extract the relevant columns from a single output file, so ran bedtools many times. That's fine, just a different way, not sure which might be fastest/most flexible?

  The sample groups can be "built" using the information in `Tco2.fqfiles`, so that can be used to direct where output goes too (thinking bams for each sample group in the same folder, one run of bedtools in each folder)

- **final processing stages**
  There were 15 different experimental groups in this dataset, with differing numbers of replicates. What they were and how many of each you could work out directly from the `Tco2.fqfiles` sample details file. As we did get more samples, the code below will still work perfectly well, but will produce different groups and replicate numbers of course!

```
while read SampleName SampleType replicate Time Treatment End1 End2
 do
 echo ${SampleType}.${Time}.${Treatment}
done < ${my_Tco_detailsfile} | tail -n +2 | sort | uniq -c > reps.groups
```

cat reps.groups

```
      3 Clone1.0.Uninduced
      3 Clone1.24.Induced
      3 Clone1.24.Uninduced
      4 Clone1.48.Induced
      3 Clone1.48.Uninduced
      3 Clone2.0.Uninduced
      3 Clone2.24.Induced
      4 Clone2.24.Uninduced
      3 Clone2.48.Induced
      3 Clone2.48.Uninduced
      3 WT.0.Uninduced
      4 WT.24.Induced
      3 WT.24.Uninduced
      3 WT.48.Induced
      3 WT.48.Uninduced
```

For calculating averages, we already know that bash isn't good at doing maths, but `awk` definitely is. If your output file contained the bed file details, and then three columns of gene expression values (for example, but you could get these values from the reps.groups file above...) saved in a file called `Clone1.0.Uninduced.genecounts`, then you could use something like:

```
while read reps groups
 do
  # bring in the variables so that awk can use them
  awk -v reps=${reps} -v groups=${groups} '{
  sum=0;
  # start at the relevant column and sum to the end of each row
  for (i=NF-reps+1; i<=NF; i++) sum+=$i;
  print sum/reps > groups".averages"
                      }' ${groups}.genecounts ;
 done < reps.groups
```

head -3 Clone1.0.Uninduced.genecounts

```
12 14 30
17 24 10
200 124 170
```

head -3 Clone1.0.Uninduced.averages

```
18.6667
17
164.667
```

Alternatively, one could use `bc` for all things maths in bash.

Your code:
Looks like it works, but no comparisons between the biological groups, sadly?! What are the biological questions being asked?
The Optional ICA said: "the statistical mean (average) of the counts per gene for each group", but not all groups in the full set were represented! What will happen when the new samples come, and they might be from clone3....??
If one of a pair of comparisons has 0 counts for a gene, that doesn't mean fold change will be zero. It might be infinite, so it is commonplace to add a very small value to make things non-zero, thereby preventing the division by/with zero issue.
You were not asked to do things in log2, but if you are interested, it's quite simple to do in awk: https://stackoverflow.com/questions/13687657/awk-and-log2-divisions

- **Were there any progress indicators?**
  Yes, lots!

- **Was there any two-way interaction with the user?**
  No. Possibilities were for user to specify where the output might go, after QC (continue or not?), choice of `bowtie2` or `hisat2` etc.

Could even ask the user what comparisons they might have wanted? There's **105** possible pairwise combinations of 15 groups...

PDF comments

This bit was even more open-ended: this was for me to try and get an idea of "how" you are thinking about the problem, and any things that might need to be clarified to you and/or others.

Were these requests for the PDF answered?

1. **gives the link for your GitHub Optional ICA1 repository (https://github.com/Bxxxxxx-2025/MyFirstPipeline)**
   Yes
2. **gives the correct password to open the *Bxxxxxx*-2025.MyFirstPipeline.tar.gz.cpt file ...!**
   Yes
3. **has an overview/flowchart that describes how the pipeline processes the data**
   Yes, but not enough detail really: what were the inputs and what were the outputs (don't just tell me to go read the scripts!!)
4. **briefly indicates why you have chosen the parameters/flags that you have for each step**
   No, nothing put here to indicate anything other than default parameters being used (and in this case, they are NOT the best, remember the big fruit salad situation...?!)
5. **indicates any things the user will have to do to make things work**
   Yes, clear instructions.
6. **highlights any difficulties, if any, that you have come across**
   No, none mentioned.
7. **indicates any additional/alternative features that you think might be beneficial to include**
   None suggested.

Any final comments

There were only a couple of points to make, which I have already commented on above. Good work, despite the code not quite making it to the end (comparisons)! I hope you find this feedback helpful and constructive!
Cheers
al